

Judge TCG — Cursor Context File (Estado Real do Monorepo)

Generated: 2026-06-06

Repository: S:-judge

Status: Produção (Judge RAG) + Plataforma de Torneios (Fases 1–5 ~80–85%)

Testes: 107/109 vitest OK | ~58 pytest plataforma (1 erro coleção) | 1000+ testes runtime enterprise

1. VISÃO GERAL DO MONOREPO

O repositório S:\tcg-judge contém **DOIS PRODUTOS** no mesmo monorepo:

1.1 Judge RAG (Produto em Produção)

- **URL:** judgetcg.com.br
- **Propósito:** Assistente de regras TCG com RAG (Retrieval-Augmented Generation)
- **Estado:** Maduro, deploy ativo, monetização via Stripe
- **Stack:** FastAPI, pgvector, SSE streaming, rate limiting, perfis de confiança por jogo

1.2 Plataforma de Torneios Multi-TCG (Fases 1–5)

- **Propósito:** Gerenciamento completo de torneios (Pokémon, Lorcana, MTG, SWU)
- **Estado:** ~80–85% implementado no backend; frontend com lacunas de rotas
- **Stack:** FastAPI + Next.js 15.1 + React 19 + Supabase + Redis

1.3 Stack Real (vs. Documentação Antiga)

Componente	Documentado	Real
Next.js	14	15.1
React	18	19
Mobile	React Native (Fase 6)	Expo/RN em apps/mobile/ (estrutura extensa, não produção)
Deploy	Docker/Vercel genérico	Render (API) + Vercel (frontend) + Supabase
Backend	FastAPI modular	FastAPI + 8500+ arquivos .py (inclui RAG, runtime enterprise)

Table 1 – continued

Componente	Documentado	Real
Migrations	5 (Fases 1–5)	23 (RAG, infractions, Stripe, analytics, etc.)
Testes	97 pytest + 109 vitest	107/109 vitest OK ; ~58 pytest plataforma (1 erro); 1000+ testes runtime

2. ESTRUTURA DO MONOREPO

```

S:\tcg-judge/
|
├─ apps/
|   ├── mobile/           # Expo/React Native (estrutura extensa, não produção)
|   ├── admin_console_v2/ # Centenas de consoles HTML (runtime enterprise)
|   └─ ...
|
├─ frontend/
|   ├── runtime_console_v3/ # Next.js 15.1 + React 19 (plataforma de torneios)
|   │   ├── src/app/       # App Router
|   │   ├── src/components/ # Componentes UI
|   │   ├── src/hooks/     # Custom hooks
|   │   ├── src/lib/       # Utilitários (inclui infractions/, game-log/)
|   │   └─ src/app/api/    # BFF proxies
|   └─ ...
|
├─ services/
|   ├── api/              # FastAPI backend
|   │   ├── app/
|   │   │   ├── main.py    # Entry point — registra todos os routers
|   │   │   ├── config/    # Configuração + validação env
|   │   │   ├── tournament/ # Core de torneios (Fase 2)
|   │   │   │   ├── engine/
|   │   │   │   │   ├── swiss.py      # Pairings Swiss
|   │   │   │   │   ├── tiebreakers.py # OMW%, GW%, OGW%
|   │   │   │   │   └─ bracket.py     # Top Cut
|   │   │   │   ├── timer.py         # Timer Redis + WS
|   │   │   │   ├── flow.py          # Orquestração ciclo de vida
|   │   │   │   └─ store.py          # Persistência
|   │   │   └─ ...
|   │   └─ tcg_adapters/            # Game Adapters (Fase 1)
|   │       ├── base.py, types.py, registry.py
|   │       ├── pokemon.py, lorcana.py, mtg.py, swu.py
|   │       ├── parsers.py, validation.py
|   │       └─ sync_mtg.py, sync_pokemon.py, sync_lorcana.py

```

```

├── players/                # Player ecosystem (Fase 3)
│   ├── store.py, rankings.py, achievements.py
│   └──
├── social/                 # Social (Fase 4)
│   ├── friendships.py, messages.py, communities.py
│   └──
├── marketplace/           # Marketplace (Fase 5)
│   ├── decklists.py, sales.py, reviews.py
│   └──
├── stores/                # Lojas verificadas (Fase 5)
│   ├── store.py, verification.py, subscriptions.py
│   └──
├── payments/              # Pagamentos (Fase 3/5)
│   ├── tournament_entry.py, payouts.py
│   └──
├── notifications/         # Notificações (Fase 4)
│   ├── service.py, push.py, email.py, sms.py
│   └──
├── overlay/               # Streaming overlay (Fase 5)
│   ├── websocket.py, templates/
│   └──
├── public_api/            # API pública (Fase 5)
│   ├── auth.py, rate_limit.py, webhooks.py
│   └──
├── admin/                 # Admin panel (Fase 4)
│   ├── dashboard.py, moderation.py, audit.py
│   └──
├── analytics/             # Analytics (Fase 3)
│   ├── organizer.py, queries.py, reports.py
│   └──
├── search/                # Busca (Fase 3)
│   └── tournaments.py
├── leagues/               # Ligas (Fase 3/4)
│   ├── scoring.py, standings.py
│   └──
├── sponsorships/          # Patrocínios (Fase 5)
│   └── sponsor.py
├── jobs/                  # Jobs (Fase 4)
│   ├── ranking_decay.py, card_sync.py, worker_main.py
│   └──
├── judge-panel/           # Painel juiz (Fase 6 — componentes frontend)
├── runtime_judge.py       # Judge RAG — PRODUTO PRINCIPAL
├── RagOrchestrator        # RAG híbrido (pgvector + BM25)
├── retrieval/             # Retrieval híbrido
├── ...
├── generated/             # Artefatos runtime enterprise (8500+ .py)

```

```

| | └─ evaluation/          # Qualidade Judge (pipelines CI)
| |   └─ ingestion/        # Ingestão de regras (PDF → chunks → embeddings)
| |
| | └─ ...
|
└─ supabase/
  └─ migrations/           # 23 migrations totais
    └─ 20260604162000_infractions.sql
    └─ 20260605100000_multi_tcg_tournament.sql
    └─ 20260605120000_tournament_lifecycle.sql
    └─ 20260605140000_player_ecosystem.sql
    └─ 20260605160000_production_social_rls.sql
    └─ 20260605180000_marketplace_streaming_api.sql
    └─ ... (mais 17 migrations RAG, Stripe, analytics, etc.)
└─ infra/
  └─ aws/                  # Infra enterprise (EKS, DR, WAF, KMS)
└─ scripts/
  └─ ingest_tcg.py         # Ingestão de regras TCG
└─ docs/
  └─ README.md, ARCHITECTURE.md, API.md, DEPLOYMENT.md, API_PUBLIC.md
  └─ PROJECT_CONTEXT.md    # Confirma: Judge RAG é o produto principal
└─ docker/
  └─ Dockerfile.worker
  └─ docker-compose.prod.yml
└─ .github/
  └─ workflows/
    └─ judge-evaluation.yml # Pipeline CI de avaliação

```

3. JUDGE RAG (PRODUTO EM PRODUÇÃO)

3.1 Visão Geral

- **URL:** judgetcg.com.br
- **Core:** `services/api/app/runtime_judge.py`, `RagOrchestrator`
- **Retrieval:** Híbrido (pgvector + BM25)
- **Streaming:** SSE (Server-Sent Events)
- **Auth:** Supabase Auth
- **Rate Limit:** Redis-based
- **Monetização:** Stripe billing, feature gates, pricing tiers

3.2 Rotas Principais

```

/judge          # Interface principal
/judge/rulings  # Consulta de regras

```

```
/judge/metrics      # Métricas de uso
/judge/dashboard    # Dashboard do usuário
/pricing            # Planos de preço
/settings/billing   # Gestão de billing
```

3.3 Componentes

```
# services/api/app/
runtime_judge.py      # Entry point do RAG
RagOrchestrator       # Orquestração RAG
retrieval/            # Módulo de retrieval híbrido
├─ pgvector_search.py # Busca semântica
├─ bm25_search.py     # Busca lexical
└─ hybrid_fusion.py   # Fusão de resultados

stripe_billing/       # Monetização
├─ pricing.py         # Planos e feature gates
├─ billing.py         # Gestão de assinaturas
└─ webhooks.py        # Webhooks Stripe
```

3.4 Infraestrutura

```
infra/aws/
├─ eks/               # Kubernetes (EKS)
├─ dr/                # Disaster Recovery
├─ waf/               # Web Application Firewall
└─ kms/               # Key Management Service
```

4. PLATAFORMA DE TORNEIOS (FASES 1–5)

4.1 Estado de Implementação

Fase	Backend	Frontend	Status
1 —Multi-TCG Foundation	✅ 100%	✅ 100%	Completo
2 —Engine Operacional	✅ 100%	✅ 100%	Completo
3 —Player Ecosystem	✅ 100%	✅ ~90%	Quase completo
4 —Produção + Social + PWA	✅ 100%	✅ ~85%	Quase completo
5 —Marketplace + Overlay + API	✅ 100%	✅ ~80%	Quase completo
6 —Mobile + IA + POS + Juiz	⚠️ Schema parcial	⚠️ Componentes mínimos	Planejado

4.2 Backend — O que está implementado

```
# main.py — routers registrados (confirmado)
tournament_system_router      # CRUD torneios
tournament_flow_router        # Ciclo de vida (check-in, rounds, standings)
tcg_adapters_router           # Game adapters (POKEMON, LORCANA, MTG, SWU)
players_router                # Perfis, rankings, achievements
social_router                  # Amigos, mensagens, comunidades
marketplace_router             # Decklists, vendas, reviews
stores_router                  # Lojas, verificação, assinaturas
payments_router                # Stripe, PIX, PayPal
notifications_router           # Push, email, SMS
overlay_router                 # OBS streaming
public_api_router              # API pública (X-API-Key)
admin_router                   # Dashboard, moderação
leagues_router                 # Ligas e standings
search_router                  # Busca de torneios
analytics_router               # Analytics do organizador
sponsorships_router            # Patrocínios
```

4.3 Frontend — Rotas Implementadas vs. Documentadas

Rota	Estado	Nota
/ (landing)	✓	TournamentShowcase
/search	✓	Descoberta de torneios
/tournament/create	✓	Wizard com GameSelector
/tournament/[id]	✓	Dashboard organizador
/tournament/[id]/play	✓	Vista do jogador
/player/[handle]	✓	Perfil público
/player/me	✗ AUSENTE	Perfil próprio (edição)
/stores	✗ AUSENTE	Listagem de lojas
/stores/[slug]	✓	Perfil da loja
/stores/create	✗ AUSENTE	Criar loja
/marketplace	✓	Listagem de decklists
/marketplace/[id]	✗ AUSENTE	Detalhe do decklist
/leagues	✓	Listagem
/leagues/create	✓	Wizard 12 semanas
/leagues/[id]	✓	Visualização
/social	✓	Comunidades + chat (agregado)
/social/friends	✗ AUSENTE	Lista separada
/social/communities	✗ AUSENTE	Lista separada
/developer	✓	Developer portal
/overlay/[tournamentId]	✓	Config OBS
/dashboard/organizer	✓	Analytics
/admin	✓	Painel admin
/pos	✗ AUSENTE	Interface POS
/judge	✓	Painel juiz (Fase 6 parcial)

Table 3 – continued

Rota	Estado	Nota
/judge/dashboard	✓	Dashboard juiz

4.4 Frontend — Lacunas de Componentes

Componentes existentes (✓):

- GameSelector, TournamentCreateForm, RoundManager, TimerDisplay
- StandingsTable, PairingsView, ResultReporter, BracketTree
- PlayerProfile, StatsCard, AchievementBadge, TournamentHistory
- RankingDisplay, DecklistCard, ReviewCard, StoreCard
- OverlayConfig, OverlayPreview, ChatDrawer, FriendButton
- MobileLayout, MobilePairingCard, OnboardingFlow, PermissionPrompt

Componentes AUSENTES (✗):

- /stores/create → StoreCreateForm
- /marketplace/[id] → DecklistDetail (parcial)
- /player/me → PlayerProfileEditForm
- /social/friends → FriendsListPage
- /social/communities → CommunitiesListPage
- /pos → POSInterface
- Fase 6: AIAssistant, JudgeDashboard (parcial em judge-panel/)

5. TESTES — ESTADO REAL

5.1 Frontend (Vitest)

Total: 109 testes
Passando: 107 ✓
Falhando: 2 ✗

Falhas:

1. OnboardingFlow.test.tsx — erro de renderização/estado
2. [Segundo teste OnboardingFlow relacionado]

5.2 Backend — Plataforma de Torneios (pytest)

~58 testes executáveis nas pastas do PDF
1 erro de COLEÇÃO em test_tournament_api.py

Erro: AsyncSession usado como response model do FastAPI

- FastAPI não aceita SQLAlchemy AsyncSession como tipo de retorno
- Corrigir: usar schema Pydantic para response model

5.3 Backend — Runtime Enterprise

1000+ arquivos de teste em services/api/generated/
Maioria relacionada a Judge RAG e runtime enterprise

Não documentados no contexto de torneios

6. DATABASE — MIGRATIONS REAIS (23 total)

6.1 Migrations da Plataforma de Torneios (5 do PDF + extensões)

```
-- Confirmadas existentes:
20260604162000_infractions.sql      -- Infrações (Fase 6 parcial)
20260605100000_multi_tcg_tournament.sql -- Fase 1
20260605120000_tournament_lifecycle.sql -- Fase 2
20260605140000_player_ecosystem.sql  -- Fase 3
20260605160000_production_social_rls.sql -- Fase 4
20260605180000_marketplace_streaming_api.sql -- Fase 5
```

6.2 Migrations Adicionais (RAG + Enterprise)

```
~17 migrations adicionais não documentadas no PDF original:
- Stripe billing, pricing, subscriptions
- RAG corpus, chunks, embeddings
- Runtime operational tables
- Analytics avançados
- Feature gates
- ... (auditar supabase/migrations/)
```

7. PROBLEMAS IDENTIFICADOS E PRIORIDADES

7.1 Críticos (Corrigir Imediatamente)

#	Problema	Impacto	Solução
1	test_tournament_api.py — AsyncSession como response model	CI quebrado	Substituir por schema Pydantic
2	2 testes vitest falhando (OnboardingFlow)	CI quebrado	Debug renderização/estado
3	Rotas frontend ausentes (/player/me, /stores, /marketplace/[id])	UX incompleta	Criar pages.tsx
4	.pem locais no repositório	Risco de segurança	Auditar e remover, adicionar a .gitignore

7.2 Altos (Próxima Sprint)

#	Problema	Prioridade
5	Fase 6 —decidir: mobile app vs. painel juiz vs. POS	Estratégico
6	Atualizar documentação (PDF) com stack real	Manutenção
7	Consolidar testes da plataforma de torneios	Qualidade
8	Deploy: Render + Vercel + Supabase configurados?	Operações

7.3 Médios (Backlog)

#	Problema	Contexto
9	apps/mobile/ —estrutura extensa, não app único	Fase 6
10	services/api/generated/ —8500+ arquivos .py	Runtime enterprise
11	apps/admin_console_v2/ —centenas de consoles HTML	Legado?
12	Infra AWS —EKS, DR, WAF, KMS documentados?	Operações

8. DECISÕES ARQUITETURAIS ATUALIZADAS

8.1 Produto Principal

- **Judge RAG** é o produto em produção (judgetcg.com.br)
- **Plataforma de Torneios** é o produto em desenvolvimento ativo
- Ambos compartilham infraestrutura (Supabase, Redis, Stripe)

8.2 Deploy

```
API (FastAPI)      → Render (render.com)
Frontend (Next.js) → Vercel (vercel.com)
Database           → Supabase (supabase.com)
Redis              → Upstash / Redis Cloud
Storage            → Supabase Storage
```

8.3 Auth

- Supabase Auth (JWT)
- RBAC: player | organizer | judge | admin
- RLS em todas as tabelas

8.4 Pagamentos

- Stripe (cartão, PIX no Brasil)
 - Feature gates por plano (Judge RAG)
 - Comissões de plataforma (torneios: 5–10%, marketplace: 30%)
-

9. CONVENÇÕES DE CÓDIGO

Python

- PEP 8, type hints obrigatórios
- Async/await para I/O
- **NUNCA** usar `AsyncSession` como response model do FastAPI
- pytest, coverage > 80%

TypeScript / Next.js

- Strict mode, App Router
- React 19 (hooks, não classes)
- Tailwind CSS
- Vitest + React Testing Library

Database

- Snake_case, UUID PKs, JSONB para flexível
- Índices para busca, RLS para segurança

Git

- Conventional commits: feat:, fix:, refactor:, test:, docs:
 - **NUNCA** commitar `.pem`, `.env`, `service-account.json`
-

10. COMANDOS ÚTEIS

```
# Backend
cd services/api
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend
cd frontend/runtime_console_v3
npm install
npm run dev          # Next.js 15.1
npm run test         # Vitest

# Tests
pytest tests/tournament/ tests/tcg_adapters/ tests/players/
pytest tests/ -k "not test_tournament_api" # Ignorar teste quebrado temporariamente
```

```
# Database
cd supabase
supabase db push
supabase db reset

# Worker
python services/api/worker_main.py

# Docker (local)
docker-compose -f docker/docker-compose.prod.yml up
```

11. CHECKLIST DE SEGURANÇA

- Remover todos os arquivos .pem do repositório
- Verificar .env* não commitados (exceto .env.example)
- Auditar service-account.json, credentials.json
- RLS ativado em TODAS as tabelas novas
- Rate limiting ativo na API pública
- Webhooks Stripe validam assinatura
- CORS configurado corretamente (produção vs. local)

12. ROADMAP ATUALIZADO

Fase	Status	Próximo Passo
1-2	✅ Completo	Manutenção
3-5	~85%	Completar rotas frontend ausentes
6	Planejado	Decidir prioridade: Mobile vs. Juiz Digital vs. POS
Judge RAG	✅ Produção	Evolução contínua
Infra	✅ AWS	Documentar runbooks

13. NOTAS PARA O CURSOR

1. Sempre verificar se o código pertence ao Judge RAG ou à Plataforma de Torneios
2. Nunca assumir que uma feature do PDF está 100% implementada — verificar rotas e componentes
3. Testes quebrados = prioridade máxima (CI/CD)
4. Segurança > features — nunca commitar credenciais
5. O monorepo tem 8500+ arquivos .py em generated/ — não tentar entender tudo, focar no contexto
6. Next.js 15.1 + React 19 — usar patterns modernos (Server Components, Server Actions)
7. Supabase é a fonte da verdade — verificar migrations antes de alterar schema

Este arquivo reflete o ESTADO REAL do monorepo em 2026-06-06. Atualizar após cada sprint ou mudança arquitetural significativa.